

PRACTICAL- 7 (A)

AIM

Game Playing in AI

a. Write a python code for implementing Min Max Algorithm

THEORY

The Minimax algorithm is a decision-making algorithm used in artificial intelligence for two-player turn-based games such as Tic-Tac-Toe, Chess, and Checkers. It works on the assumption that one player tries to maximize their score while the opponent tries to minimize it. The algorithm explores all possible game states by building a game tree and evaluates the best move by recursively calculating the optimal value for each node. The maximizing player selects the move with the highest value while the minimizing player selects the move with the lowest value. By analysing all possible future moves, the Minimax algorithm ensures that the best possible decision is made assuming the opponent also plays optimally.

SOFTWARE USED

Jupyter Notebook

CODE

```
import math
```

```
def minimax(depth, node_index, is_max, scores, height):  
    if depth == height:  
        return scores[node_index]  
  
    if is_max:  
        return max(minimax(depth+1, node_index*2, False, scores, height),  
                    minimax(depth+1, node_index*2+1, False, scores, height))  
    else:  
        return min(minimax(depth+1, node_index*2, True, scores, height),  
                    minimax(depth+1, node_index*2+1, True, scores, height))
```

```
scores = [3,5,6,9,1,2,0,-1]
tree_depth = int(math.log2(len(scores)))

print(minimax(0,0,True,scores,tree_depth))
```

OUTPUT

```
print(minimax(0,0,True,scores,tree_depth))
5
```

PRACTICAL- 7 (B)

AIM

Game Playing in AI

b. Write a python code for implementing Alpha Beta Pruning Algorithm

THEORY

Alpha-Beta Pruning is an optimization technique used with the Minimax algorithm to improve its efficiency by reducing the number of nodes evaluated in the game tree. It works by eliminating branches that cannot possibly affect the final decision of the game. Two parameters called alpha and beta are used to track the best values for the maximizing and minimizing players. Alpha represents the best value that the maximizing player can guarantee, while beta represents the best value that the minimizing player can guarantee. When the algorithm detects that exploring a branch will not produce a better result than the current best value, that branch is pruned. This significantly reduces the computational complexity and allows deeper searches in complex games.

SOFTWARE USED

Jupyter Notebook

CODE

```
import math
```

```
def alphabeta(depth, node_index, is_max, scores, height, alpha, beta):  
    if depth == height:  
        return scores[node_index]  
  
    if is_max:  
        best = -math.inf  
        for i in range(2):  
            val = alphabeta(depth+1, node_index*2+i, False, scores, height, alpha, beta)  
            best = max(best, val)  
        alpha = max(alpha, best)
```

```
        if beta <= alpha:
            break
    return best
else:
    best = math.inf
    for i in range(2):
        val = alphabeta(depth+1, node_index*2+i, True, scores, height, alpha, beta)
        best = min(best, val)
        beta = min(beta, best)
        if beta <= alpha:
            break
    return best

scores = [3,5,6,9,1,2,0,-1]
tree_depth = int(math.log2(len(scores)))

print("Alpha-beta-pruning output: ",alphabeta(0,0,True,scores,tree_depth,-math.inf,math.inf))
```

OUTPUT

```
Alpha-beta-pruning output: 5
```

PRACTICAL- 7 (C)

AIM

Game Playing in AI

c. Write a python code for implementing a game using minmax with Alpha Beta Pruning.

THEORY

Game playing is an important application of artificial intelligence where algorithms are used to simulate intelligent decision making in competitive environments. In such games, the AI must analyse possible moves and choose the best one based on predicted outcomes. The Minimax algorithm is commonly used for this purpose as it evaluates all possible future moves and selects the optimal strategy assuming the opponent also plays optimally. However, exploring all possible game states can be computationally expensive. Alpha-Beta pruning improves the efficiency of the Minimax algorithm by eliminating branches that cannot influence the final decision. This allows the AI to search deeper in the game tree and make faster decisions. When applied to games like Tic-Tac-Toe, this approach enables the AI to play optimally and either win or force a draw against a human player.

SOFTWARE USED

Jupyter Notebook

CODE

```
import math
```

```
board = [" " for _ in range(9)]
```

```
def check_winner(b, player):
```

```
    wins = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
```

```
    for i in wins:
```

```
        if b[i[0]] == b[i[1]] == b[i[2]] == player:
```

```
            return True
```

```
    return False
```

```
def is_full(b):
```

```
return " " not in b
```

```
def minimax(b, depth, is_max, alpha, beta):
```

```
    if check_winner(b,"O"):
```

```
        return 1
```

```
    if check_winner(b,"X"):
```

```
        return -1
```

```
    if is_full(b):
```

```
        return 0
```

```
    if is_max:
```

```
        best = -math.inf
```

```
        for i in range(9):
```

```
            if b[i] == " ":
```

```
                b[i] = "O"
```

```
                val = minimax(b, depth+1, False, alpha, beta)
```

```
                b[i] = " "
```

```
                best = max(best, val)
```

```
                alpha = max(alpha, best)
```

```
                if beta <= alpha:
```

```
                    break
```

```
        return best
```

```
    else:
```

```
        best = math.inf
```

```
        for i in range(9):
```

```
            if b[i] == " ":
```

```
                b[i] = "X"
```

```
                val = minimax(b, depth+1, True, alpha, beta)
```

```
                b[i] = " "
```

```
        best = min(best, val)

        beta = min(beta, best)

        if beta <= alpha:

            break

    return best
```

```
def best_move():

    best_val = -math.inf

    move = -1

    for i in range(9):

        if board[i] == " ":

            board[i] = "O"

            move_val = minimax(board,0,False,-math.inf,math.inf)

            board[i] = " "

            if move_val > best_val:

                best_val = move_val

                move = i

    return move
```

```
def print_board():

    for i in range(0,9,3):

        print(board[i],board[i+1],board[i+2])
```

```
while True:

    print_board()

    user = int(input("Enter position (0-8): "))

    if board[user] == " ":

        board[user] = "X"

    if check_winner(board,"X"):
```

```

    print_board()
    print("You Win")
    break
if is_full(board):
    print_board()
    print("Draw")
    break

ai = best_move()
board[ai] = "O"

if check_winner(board,"O"):
    print_board()
    print("AI Wins")
    break
if is_full(board):
    print_board()
    print("Draw")
    break

```

OUTPUT

```

Enter position (0-8): 0
X
 0

Enter position (0-8): 2
X 0 X
 0

Enter position (0-8): 7
X 0 X
0 0
 X

Enter position (0-8): 5
X 0 X
0 0 X
 X 0

Enter position (0-8): 6
X 0 X
0 0 X
X X 0
Draw

```


